Lawful Composition Calculus (LCC) + Associator Spectroscopy — v0.2

A practical algebra + diagnostics stack for composition under interaction laws

---

0) Executive summary

We formalize composition **with laws baked in** and pair it with an empirical diagnostic for closure. The algebraic side is the **Lawful Composition Calculus (LCC)**: a law-shaped product $\bowtie_K$ that internalizes interaction constraints. The diagnostic side is **associator spectroscopy**: a measured gap that lights up precisely when macro-composition forgets information relevant to the next step. A small **information-bottleneck (IB)** learner then discovers the *minimal augmentation* that restores closure.

Key deliverables:

- **Algebra:** reusable $\bowtie_K$ composition (subobject or quotient form) and provable submultiplicativity for a family of size measures.
- **Theorem:** *macro associativity ⇔ sufficiency for bracketing*.
- **Diagnostics:** associator gap Δ with visualization + a parsimony-controlled IB closure procedure.
- **Implementations:** deterministic & probabilistic APIs; constraint generators (no full cartesian blowups); temporal extension.

> **Naming note.** We replace "constraint-coupled tensor" with **Lawful Composition Calculus (LCC)** and the operator $\otimes_K$ with $\bowtie_K$ (join under law K) to avoid multilinearity baggage while keeping the right intuition (fibered product / constrained pullback).

---

1) Core objects and composition

1.1 Lawful system

A **Lawful System** is a triple `(X, α, μ)`

- `X`: state space (discrete, continuous, or probabilistic support),
- `α: X→X`: *consistency* map (idempotent when intrinsic law is a projector; otherwise treat as an iterative refinement operator),
- `μ`: a size/measure (counting, volume, or probability measure).

> If the domain lacks a raw/consistent distinction, set `α = id`.

1.2 Interaction law

A **law** is a relation `K ⊆ X×Y` that is `α`-congruent: `(x,y)∈K ⇒ (α_X(x), α_Y(y))∈K`.

1.3 Lawful composition $\bowtie_k$

Two equivalent realizations; pick per application:

- **Subobject form (selection):** `X ⋈ₖ Y := {(x,y)∈X×Y : (x,y)∈K}` with `α_{X⋈Y} := α_X×α_Y`.
- **Quotient form (identifications):** equip `X×Y` with the smallest congruence `~ₖ` generated by `K` and `α` -identifications, then set `X ⋈ₖ Y := (X×Y)/~ₖ`.

Symmetry/unit are properties of `K` (don't assume them). Expose them as capabilities once verified.

---

2) Size / information measures & submultiplicativity

Let `Fix(α_X) := {x : α_X(x)=x}` and define size/information:

- **Discrete:** `S(X) := μ(Fix(α_X))` (counting),
- **Continuous:** `S(X) := μ(Fix(α_X))` (volume/measure),
- **Robust:** `S_ε(X) := μ({x : d(α_X(x),x)≤ε})`,
- **Informational:** `H(X) := H( distribution on Fix(α_X) )`.

Then universally:

> **Submultiplicativity.** `S(X ⋈ₖ Y) ≤ S(X)·S(Y)` and likewise for `S_ε` and `H` (by data-processing / measure subadditivity). Equality characterizations depend on subobject vs quotient form but always coincide with: *every lawful pair survives and no extra identifications collapse lawful pairs*.

---

3) Macro semantics & the closure theorem

Let `⊙` be an associative **micro** operation (relational join / monoidal convolution) and let `Φ` be a **coarse-graining** (deterministic map or Markov kernel). Define the **macro** operation by pushforward/disintegration:

`x ⊕_Φ y := Φ_* ( x ⊙ y | Φ )`.

Define the **associator gap** for a divergence `D` (TV, KL, Wasserstein, MSE):

`Δ_Φ(x,y,z) := D( (x ⊕_Φ y) ⊕_Φ z , x ⊕_Φ (y ⊕_Φ z) )`.

> **Lawful Composition Closure Theorem.**
>
> 1. (**Closure**) `Δ_Φ ≡ 0` **iff** there exists a deterministic operation `⊕̂` on macro-states with `Φ(x ⊙ y) = Φ(x) ⊕̂ Φ(y)` for all lawful pairs. Equivalently: `Φ` is a **sufficient statistic for bracketing**.

2. (**ε-closure via IB**) The minimal augmentation `C` such that `Δ_{Φ,C} ≤ ε` is an `ε`-approximate sufficient statistic. A principled search is the **information bottleneck**: minimize `E[Δ_{Φ,C}] + β·I(C; micro)`.
3. (**Compositionality**) For clause libraries `(K = K₁ ∩ K₂)`, `Δ_Φ ≤ Δ_{Φ,K₁} + Δ_{Φ,K₂}` (triangle inequality at the diagnostic level).

Proof sketch: disintegrate both bracketings; equality for all triples implies a unique macro law; conversely a macro law commuting with `Φ` collapses both bracketings. IB is standard sufficiency under a rate constraint.

---

4) Learning the minimal augmentation `C`

We learn `C` (bit flags, small embeddings, decision lists) via:

`min_C  E[Δ_{Φ,C}] + β·I(C; micro)` subject to `Ψ=(Φ,C)`.

- **Parsimony:** sweep `β` to trace a Pareto frontier (gap vs bits).
- **Validation:** nested CV on triples; report held-out `\bar{Δ}` with CIs.

Law card selection. If `K = ∩_i K_i` with per-clause penalties, choose a sparse subset or a tiny BDD/automaton; ablations show which clauses are empirically necessary.

---

5) Temporal extension

For trajectories `x_{0:T}`, allow time-indexed constraints `K_t` or pathwise `K_path ⊆ X^T`. Define:

`X ⋈_{K[0:T]} Y := { (x_{0:T}, y_{0:T}) : (x_t, y_t) ∈ K_t ∀ t }`.

Spectroscopy scans `Δ_Φ` across triple blocks and lags; the learned augmentation often collapses to *finite memory* (last-merge scale, boundary tag, etc.).

---

6) Implementation notes (ready-to-code)

6.1 Discrete/probabilistic API skeleton

```python
class LawfulSystem:
    def __init__(self, states, alpha=lambda x: x, measure=None):
        self.states, self.alpha = states, alpha
        self.measure = measure  # default: counting
    def lawful(self):
        return {x for x in self.states if self.alpha(x)==x}
```

```
class Constraint:
    def __init__(self, name, predicate=None, generator=None):
        self.name, self.pred, self.gen = name, predicate, generator
    def __call__(self, x, y):
        return self.pred(x, y)
    def pairs(self, A, B):  # generator to avoid |A|·|B| blowups
        yield from self.gen(A, B)
    def __and__(self, other):
        def new_gen(A,B):
            for a,b in self.gen(A,B):
                if other.pred(a,b):
                    yield (a,b)
        return Constraint(f"({self.name}&{other.name})", predicate=lambda a,b:
self(a,b) and other(a,b), generator=new_gen)
```

6.2 Constraint generators

Compile `K` to a **predicate** on features, a **BDD/automaton**, or a **GPU mask**—never materialize full cartesian products. Sampling + concentration bounds can certify closure without full enumeration.

6.3 Macro + reconstruction

- Coarse-grain `Φ` (keep/average/cluster/Markov kernel) then reconstruct the forgotten variable with **constraint-aware maximum entropy**; this is the law-respecting baseline.
- Deviations (biases, heuristics) expose `Δ_Φ` spikes and guide augmentation learning.

---

7) Mini-experiments (implemented)

Bool + nearly-diagonal law. With constraint-aware reconstruction, `Δ≈0` (macro is sufficient). Inject a small cross-dependence into reconstruction; `Δ` rises smoothly with bias strength `ε`, cleanly demonstrating closure failure and the **dose–response** of the associator.

Ternary mod-3 law. Using an additive macro `(x+z) mod 3`, the law-respecting reconstructor again yields `Δ≈0`; biased reconstruction reintroduces a measurable gap.

Artifacts present: data tables for left/right macro distributions and **TV-gap curves** from the bias sweeps; plots show monotone growth of `Δ` as ε increases.

---

8) Benchmark suite (recommended)

- **Algebraic:** Bool & modular arithmetic with clause libraries (diagonal, anti-diagonal, banded).
- **Stochastic:** finite-state Markov chains with state aggregation; Wasserstein/KL variants of `Δ`.

- **Systems:** resource pooling / load consolidation (merge order effects), clustering (merge statistics), and typed protocol composition.

KPIs: closure certificates ($\Delta \leq \varepsilon$), bits of $\boxed{C}$, law-card sparsity, and runtime (triples scanned vs certified).

---

9) Documentation package

- 1-page **cheat sheet** (definitions & theorem).
- 3-page **methods note** (proof sketch, optimization objective, diagnostics).
- Interactive **notebook** (sliders for ε, β; heatmaps for Δ; law-card ablations).

---

10) External references (selected)

- **Information bottleneck & DPI**
  Tishby, Pereira, Bialek (2000), *The Information Bottleneck Method*, arXiv\:physics/0004057.
  Cover & Thomas (2006), *Elements of Information Theory*, 2nd ed., Wiley (data-processing inequality; divergences).

- **Sufficiency / comparison of experiments**
  Blackwell (1953), *Equivalent Comparisons of Experiments*, Ann. Math. Stat. 24(2):265–272.
  Le Cam (1996), *Comparison of Experiments: A Short Review*, Statist. Sci. 11(4):803–806.

- **Monoidal/compositional foundations**
  Mac Lane (1998/1971), *Categories for the Working Mathematician*, 2nd ed., Springer (monoidal categories, associators, coherence).
  Abiteboul, Hull, Vianu (1995), *Foundations of Databases*, Addison–Wesley (natural join; associativity/commutativity).

- **Metrics for macro comparisons**
  Villani (2008), *Optimal Transport: Old and New*, Springer (Wasserstein distances and geometry).

(Full links are available on request; these are standard texts/articles.)

---

11) Roadmap

Now: package the reference implementation with plots + a tiny IB learner; add law-card ablation utilities.
Next: temporal benchmarks and Wasserstein/KL diagnostics; export a “closure certificate” artifact.
Later: categorical packaging (monoidal-with-constraints), typed protocols, and case studies (resource pooling; clustering; program composition).

Appendix A — Minimal augmentation IB objective

`min_C  E[Δ_{Φ,C}] + β·I(C; micro)` with nested CV; report Pareto frontier (gap vs bits) and selected `C`.

Appendix B — Equality conditions

- **Subobject form:** equality $S(X \bowtie_k Y)=S(X)S(Y)$ iff all lawful pairs survive (`Fix(α_X)×Fix(α_Y) ⊆ K`).
- **Quotient form:** additionally require that $\sim_k$ **does not** identify distinct lawful pairs.

Appendix C — Engineering checklist

- Pick form (subobject vs quotient) per domain.
- Compile `K` to generator/BDD; certify `α`-congruence.
- Choose `S` and divergence `D` (TV/KL/Wasserstein).
- Design `Φ` (and constraint-aware reconstruction).
- Run spectroscopy, learn minimal `C`, export closure card + law card.